

SnapVM: Snapshot-Based MicroVM Environments for Fast Execution and Recovery

Arthur Carvalho Rodrigues  [Pontifícia Universidade Católica de Minas Gerais | arthurcarvalhorodrigues2409@gmail.com]

Débora Luiza de Paula Silva  [Pontifícia Universidade Católica de Minas Gerais | debora.paula05@gmail.com]

Gustavo Henrique Rodrigues de Castro  [Pontifícia Universidade Católica de Minas Gerais | guguh1612@gmail.com]

Mariana Almeida Mendonça  [Pontifícia Universidade Católica de Minas Gerais | marianaalmeidaafa@gmail.com]

✉ Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500 – Coração Eucarístico, Belo Horizonte – MG, 30535-901, Brazil.

Resumo. Este trabalho apresenta o SnapVM, um sistema experimental de orquestração de ambientes isolados baseado em microVMs e snapshots. Diferentemente de abordagens que tratam snapshots apenas como mecanismo de infraestrutura, este artigo investiga seu uso como primitiva operacional para agentes de IA em ambientes *stateful*. A proposta parte do problema de que agentes baseados em ferramentas podem modificar bancos de dados, processos, arquivos e configurações temporárias, tornando falhas difíceis de reverter apenas com mecanismos de rollback lógico. O SnapVM expõe operações como restauração de snapshot e checkpointing ao ciclo de decisão do agente, permitindo retornar a estados confiáveis, reduzir reparos manuais e preservar a continuidade de tarefas complexas. Foram conduzidos experimentos reais e preliminares com agente ReAct baseado em GPT-4o, avaliando consumo de tokens, latência, chamadas de ferramenta, poluição de contexto e taxa de sucesso. Os resultados indicam que snapshots podem reduzir a carga cognitiva do agente e funcionar como uma forma de memória operacional reversível, embora o projeto ainda esteja em evolução e demande validações adicionais.

Keywords: MicroVM; Firecracker; Snapshots; Virtualização Leve; Execução Isolada; Serverless; Agentes de IA; Checkpointing;

© Published under the Creative Commons Attribution 4.0 International Public License (CC BY 4.0)

1 Introdução

Agentes de IA contemporâneos vêm deixando de atuar apenas como geradores de texto para operar como sistemas capazes de executar ferramentas, consultar bancos de dados, modificar arquivos, rodar comandos de terminal e validar aplicações em ciclos iterativos de ação e observação. Esse padrão de uso torna os agentes particularmente úteis em tarefas de desenvolvimento de software, automação, testes, depuração e operação de ambientes computacionais. Entretanto, quando um agente atua sobre um ambiente *stateful*, seus erros não permanecem restritos ao texto produzido: uma decisão incorreta pode remover tabelas, corromper dados, alterar permissões, derrubar processos, modificar arquivos de configuração ou deixar a aplicação em um estado intermediário difícil de reconstruir.

Ferramentas tradicionais de versionamento, como o Git, são adequadas para restaurar arquivos e código-fonte, mas não têm como objetivo recuperar o estado vivo de processos, bancos de dados, descritores abertos, memória e configurações transitórias. Por esse motivo, neste trabalho o Git é utilizado como um **baseline mínimo de rollback lógico**, e não como concorrente direto de mecanismos de snapshot. O interesse central é avaliar o custo adicional imposto ao agente quando apenas o sistema de arquivos é revertido, mas o ambiente de execução permanece parcialmente corrompido. Nesses casos, o agente precisa consumir tokens e tempo para diagnosticar, formular hipóteses, executar comandos corre-

tivos e verificar se o sistema voltou a funcionar.

A computação em nuvem moderna, especialmente no modelo serverless, já evidenciou a necessidade de conciliar isolamento forte e baixo overhead. Agache *et al.* [2020], ao descreverem a motivação para o desenvolvimento do Firecracker na AWS, sintetizaram esse dilema como a tensão entre virtualização com forte segurança e alto overhead, e tecnologias de container com menor overhead, porém isolamento mais fraco. Containers, como o Docker, usam namespaces e cgroups do Linux para isolar processos, mas compartilham o kernel do hospedeiro; conforme analisado por Jain [2020] e por Weissman *et al.* [2023], essa dependência cria riscos relevantes quando se executa código não confiável. Máquinas virtuais tradicionais, por sua vez, oferecem isolamento mais robusto, mas apresentam custo de inicialização e memória maior, especialmente quando baseadas em hipervisores completos como QEMU/KVM [Agache *et al.*, 2020].

O Firecracker surgiu como uma resposta direta a esse dilema. Desenvolvido pela AWS e apresentado no NSDI 2020 [Agache *et al.*, 2020], ele é um Virtual Machine Monitor (VMM) escrito em Rust que utiliza o KVM do Linux para executar microVMs com footprint reduzido e inicialização rápida. Ao eliminar emulação de dispositivos legados e expor uma API REST minimalista via socket local, o Firecracker fornece uma base técnica interessante para ambientes isolados, leves e restauráveis. Ainda assim, o Firecracker é uma peça de infraestrutura de baixo nível: criar

snapshots, restaurá-los, registrar metadados, expor ferramentas para agentes e organizar experimentos exige uma camada adicional de orquestração. É nessa lacuna que se posiciona o SnapVM.

O SnapVM investiga snapshots de ambientes isolados como uma **primitiva operacional para agentes de IA**. Nesse modelo, operações como `save_checkpoint`, `restore_checkpoint` e `restore_last_snapshot` deixam de ser apenas rotinas internas de infraestrutura e passam a compor o espaço de ações do agente. A hipótese deste trabalho é que snapshots podem reduzir a carga cognitiva do agente em ambientes *stateful*, aproximando a restauração de um custo estável em cenários controlados e transferindo parte do esforço de recuperação do raciocínio textual para uma ação ambiental atômica.

Os resultados apresentados neste artigo são **reais, porém preliminares**, pois o projeto segue em evolução. Os experimentos não foram conduzidos como testes únicos: o Experimento 2 utilizou 20 iterações por baseline; o Experimento 4 utilizou 20 execuções por baseline; os Experimentos 4.1 e 4.2 avaliaram cinco cenários de falhas com quatro repetições por cenário; e o Experimento 5 executou três trials por baseline. De modo geral, os dados indicam reduções em consumo de tokens, latência de recuperação e poluição de contexto quando snapshots são disponibilizados como ferramentas para o agente.

1.1 Contribuições

As principais contribuições deste trabalho são:

- propor snapshots como primitiva operacional para agentes de IA em ambientes *stateful*;
- apresentar o SnapVM como camada experimental que expõe restauração e checkpointing ao ciclo de decisão de agentes;
- avaliar preliminarmente o impacto da abordagem em consumo de tokens, latência, chamadas de ferramenta, poluição de contexto e taxa de sucesso;
- discutir a diferença entre eficiência de infraestrutura e comportamento autônomo do agente;
- apontar caminhos futuros para branching especulativo, políticas de decisão custo-aware e exploração de múltiplos estados.

1.2 Hipótese do Trabalho

A hipótese central deste trabalho é que snapshots de ambientes isolados podem reduzir a carga cognitiva de agentes de IA em tarefas *stateful*, pois transformam a recuperação de falhas em uma ação operacional explícita. Em vez de depender exclusivamente de diagnóstico textual, comandos corretivos e reconstrução manual de estado, o agente pode retornar a um ponto previamente validado. De forma mais cautelosa, não afirmamos que a restauração possui custo constante em sentido absoluto; afirmamos que, em cenários controlados e de tamanho fixo, a restauração por snapshot tende a apresentar custo mais estável que o reparo manual, cuja complexidade cresce conforme a gravidade e a opacidade da falha.

1.3 Objetivos

Os objetivos do trabalho são:

1. Projetar e implementar uma ferramenta experimental que gerencie ambientes isolados baseados em microVMs ao longo de seu ciclo de vida.
2. Expor criação, restauração e checkpointing de snapshots como operações primárias disponíveis ao agente, não apenas como detalhes de infraestrutura.
3. Avaliar se snapshots reduzem consumo de tokens, latência, chamadas de ferramenta e poluição de contexto em cenários de recuperação *stateful*.
4. Investigar se agentes conseguem escolher restauração por snapshot de forma autônoma e em quais cenários preferem reparo manual.
5. Avaliar preliminarmente checkpoints proativos como mecanismo de continuidade, preservação de progresso e memória operacional reversível.
6. Estabelecer métricas de referência para trabalhos futuros envolvendo branching especulativo, múltiplos modelos e workloads mais complexos.

1.4 Escopo e Limitações Iniciais

O SnapVM é um sistema experimental voltado para ambientes host baseados em Linux em hardware x86_64 com suporte a KVM. O projeto não tem como objetivo substituir hipervisores de produção ou runtimes de containers de propósito geral. Redes, armazenamento persistente distribuído e execução multi-host estão fora do escopo da fase atual. A contribuição principal, nesta etapa, está na análise do papel de snapshots como ferramenta de raciocínio e recuperação para agentes de IA, não na criação de um novo hipervisor.

2 Fundamentação Teórica e Trabalhos Relacionados

O design do SnapVM se apoia em múltiplas áreas: agentes de IA baseados em ferramentas, ambientes *stateful*, virtualização leve, mecanismos de snapshot e checkpoint, execução de código em sandbox e reprodutibilidade computacional. Esta seção preserva a fundamentação técnica do trabalho original, mas reorganiza os conceitos para sustentar a tese de que snapshots podem compor o ciclo de decisão de agentes.

2.1 Agentes de IA Baseados em Ferramentas

Agentes baseados em modelos de linguagem podem ser estruturados como loops de raciocínio e ação, nos quais o modelo observa um estado, escolhe uma ferramenta, executa uma ação e interpreta o resultado retornado. Em tarefas de software, essas ferramentas frequentemente incluem execução de comandos, consultas a banco de dados, leitura de logs, escrita de arquivos e validações automatizadas. O desafio surge quando cada ação do agente modifica o ambiente de forma persistente: uma consulta SQL incorreta, uma migração mal aplicada ou uma alteração de permissões podem degradar o estado global da aplicação.

Nesse contexto, a confiabilidade do agente depende não apenas de sua capacidade textual, mas também da infraestrutura que o cerca. Se o ambiente não oferece mecanismos de retorno a estados confiáveis, o agente precisa corrigir manualmente suas próprias falhas. Esse processo aumenta o número de tokens, acumula logs e erros na janela de contexto e pode levar a ciclos de tentativa e erro.

2.2 Ambientes Stateful e Recuperação de Estado

Ambientes de desenvolvimento e teste raramente são compostos apenas por arquivos versionados. Aplicações modernas envolvem bancos de dados, processos, caches, sockets, permissões, variáveis de ambiente e estados transitórios em memória. Assim, um mecanismo de rollback restrito ao sistema de arquivos não é suficiente para restaurar completamente a aplicação após uma falha. O SnapVM parte dessa observação para investigar uma abordagem baseada em captura e restauração integral do estado de execução.

2.3 Rollback Lógico, Snapshots e Checkpointing

O Git é uma ferramenta consolidada para versionamento de código-fonte, mas sua unidade de controle é o arquivo versionado. Em nossos experimentos, ele é usado como base-line mínimo de rollback lógico: ao executar `git reset --hard`, o código e as migrações no disco podem retornar ao estado anterior, mas o banco de dados e os processos já afetados permanecem alterados. Esse contraste é útil para medir a carga adicional que recai sobre o agente quando o estado vivo do ambiente não é restaurado.

Mecanismos de snapshot e checkpoint foram amplamente estudados em virtualização, migração ao vivo e tolerância a falhas. O CRIU (*Checkpoint/Restore In Userspace*) permite checkpointing de processos Linux sem suporte direto de virtualização de hardware, capturando descritores de arquivo, mapas de memória e estado de CPU [CRIU Project, 2012]. Embora poderoso, ele opera na granularidade de processos e não oferece, por si só, a fronteira completa de isolamento de uma microVM. Já pesquisas sobre bifurcação de VMs no nível do hipervisor, como SnowFlock [Lagar-Cavilla *et al.*, 2009], demonstraram que múltiplas VMs podem derivar de um estado pai usando páginas de memória com cópia-na-escrita. O SnapVM se inspira nesse princípio, mas atua como camada de orquestração de nível mais alto.

2.4 Sandboxes, Containers e MicroVMs

Containers, como Docker e Podman, resolvem parte do problema de empacotamento e isolamento ao usar namespaces e cgroups do Linux. Todavia, por compartilharem o kernel do hospedeiro, não fornecem a mesma fronteira de isolamento de uma VM completa, especialmente em cenários com código não confiável ou agentes autônomos [Weissman *et al.*, 2023]. O gVisor [Young *et al.*, 2019] oferece uma abordagem alternativa ao interceptar chamadas de sistema via um kernel em espaço de usuário; apesar do isolamento adicional,

estudos apontam overhead mensurável em cargas intensivas em syscalls [Long *et al.*, 2020].

O Kata Containers [Kata Containers Contributors, 2017] combina semântica de containers com isolamento de VM, usando QEMU ou Firecracker como VMM subjacente. Esse tipo de abordagem demonstra que containers apoiados por VMs são operacionalmente viáveis, embora priorizem compatibilidade OCI em vez de expor snapshots como primitiva central. O Firecracker [Agache *et al.*, 2020; Jain, 2020], por sua vez, fornece uma base técnica leve e isolada, adequada para cenários em que o custo de inicialização e restauração deve ser reduzido.

2.5 Execução de Código em Sandbox e Ambientes Agentic

Plataformas de execução de código como Judge0, Sphere Engine e Replit operacionalizaram sandboxes em escala, frequentemente com namespaces, cgroups e filtragem seccomp. Plataformas mais recentes, como o E2B [E2B, 2023], adotaram Firecracker como backend para oferecer isolamento mais robusto a ambientes de execução sob demanda. O SnapVM se diferencia por focar em orquestração local, auto-hospedada e centrada em snapshots como ações disponíveis para agentes.

A reprodutibilidade em ambientes computacionais também é explorada por ferramentas como Nix, Guix e Docker BuildKit, que buscam derivar ambientes a partir de especificações determinísticas. O SnapVM propõe uma abordagem complementar: em vez de reconstruir o ambiente a partir de uma especificação, captura e restaura diretamente um estado inicializado e verificado. Essa diferença é particularmente relevante para agentes de IA, pois permite retornar rapidamente a um ponto operacional confiável durante a execução de uma tarefa.

2.6 Lacuna Identificada

A literatura e as ferramentas existentes tratam snapshots principalmente como mecanismos de infraestrutura, recuperação ou migração. A lacuna investigada neste trabalho é o uso de snapshots como **ferramentas no ciclo de decisão do agente**. Em vez de apenas executar comandos e corrigir falhas manualmente, o agente passa a manipular estados do ambiente: salvar, restaurar, repetir e futuramente ramificar alternativas. Essa mudança desloca snapshots do domínio exclusivo da infraestrutura para o domínio da arquitetura agentic.

3 SnapVM: Snapshots como Primitiva para Agentes

O SnapVM é uma camada experimental de orquestração que utiliza microVMs e snapshots para fornecer ambientes isolados, restauráveis e programaticamente controláveis. Seu objetivo não é substituir o Firecracker, mas construir sobre ele uma interface de maior nível, capaz de expor operações de estado a agentes de IA.

3.1 Visão Geral da Proposta

A proposta central do SnapVM é tratar snapshots como uma primitiva de raciocínio operacional. Quando um agente enfrenta uma falha, ele não precisa necessariamente diagnosticar e reparar todos os efeitos colaterais manualmente; pode retornar a um estado anterior conhecido e tentar novamente. Quando antecipa uma ação arriscada, pode salvar um checkpoint antes de executá-la. Quando diferentes caminhos de solução forem possíveis, trabalhos futuros poderão explorar múltiplas ramificações a partir de um mesmo estado.

- **Restauração reativa:** o agente usa `restore_last_snapshot` após observar falha ou degradação do ambiente.
- **Checkpointing proativo:** o agente usa `save_checkpoint` antes de uma etapa considerada arriscada.
- **Repetição controlada:** o agente restaura um estado confiável e tenta uma abordagem alternativa.
- **Branching futuro:** o agente poderá explorar caminhos diferentes a partir do mesmo snapshot base.

3.2 Ciclo Agentic com Snapshots

O ciclo tradicional de agentes baseados em ferramentas pode ser descrito como *Reason* → *Act* → *Observe*. No SnapVM, esse ciclo é ampliado por operações de estado:

Reason → **Save** → **Act** → **Observe** → **Restore/Retry**

Essa extensão permite que o ambiente funcione como uma memória operacional reversível. O agente não depende apenas de lembrar, no contexto textual, quais ações foram realizadas; ele pode retornar fisicamente a um estado anterior do sistema.

3.3 Ferramentas Expostas ao Agente

Nos experimentos, o agente opera com um conjunto de ferramentas de alto nível, resumidas na Tabela 1. Essas ferramentas são suficientes para simular tarefas de desenvolvimento e recuperação em uma API com PostgreSQL.

Table 1. Ferramentas expostas ao agente nos experimentos

Ferramenta	Tipo	Função no ciclo agentic
<code>query_db(sql)</code>	Ação	Executa consultas ou comandos SQL no PostgreSQL do ambiente guest.
<code>execute_bash(command)</code>	Ação	Executa comandos shell no ambiente controlado pelo orquestrador.
<code>check_health()</code>	Observação	Verifica o endpoint <code>/health</code> e valida se o contrato de estado ainda é satisfeito.
<code>restore_last_snapshot()</code>	Restauração	Retorna o ambiente ao último snapshot saudável disponível.
<code>save_checkpoint(label)</code>	Checkpoint	Salva um ponto de retorno nomeado antes de etapas arriscadas.
<code>restore_checkpoint(label)</code>	Restauração	Restaura um checkpoint específico criado durante a tarefa.

3.4 Arquitetura Conceitual

O fluxo de orquestração do SnapVM mantém o diagrama técnico original do projeto, representado na Figura 1. O agente envia ações ao orquestrador Python, que traduz as decisões

em comandos para a API REST local do Firecracker. A microVM executa a aplicação, o banco de dados e os processos necessários para o experimento. Snapshots são capturados e restaurados por meio da camada de orquestração.



Figure 1. Diagrama de blocos do fluxo de orquestração do SnapVM: A interação entre a lógica agentic e a infraestrutura física de MicroVMs.

3.5 Implementação Atual do MVP

Até o momento, o projeto implementou um MVP experimental com os seguintes elementos:

- orquestrador Python responsável por controlar o ciclo de vida dos experimentos;
- integração com agente ReAct baseado em GPT-4o com temperatura 0;
- ferramentas para consulta ao banco, execução de comandos e verificação de saúde;
- baseline com Git para rollback lógico mínimo;
- restauração de snapshots para recuperação de estado do ambiente;
- experimentos com falhas simples, falhas multicamadas e checkpointing proativo;
- coleta de métricas de tokens, latência, chamadas de ferramenta, poluição de contexto e taxa de sucesso.

3.6 Stack Tecnológico

As decisões tecnológicas que fundamentam a implementação do SnapVM são apresentadas na Tabela 2. A tabela preserva a organização original do projeto e explicita a função de cada componente.

Table 2. Stack tecnológico do SnapVM

Componente	Tecnologia	Justificativa
VMM	Firecracker v1.x	Footprint mínimo, KVM, API REST, suporte a snapshot
SO Host	Linux (Ubuntu 22.04+)	Disponibilidade de KVM, suporte a cgroup v2
Linguagem de Orquestração	Python 3	Ecossistema para integração de IA e automação
Kernel Guest	Linux 5.10 (config microvm)	Boot rápido, conjunto de drivers reduzido
Rootfs/Ambiente Guest	Alpine Linux (ext4), Node.js e Bash	Tamanho mínimo (~10 MB) e suporte à API testada
IPC / API	REST Firecracker (Unix socket)	Controle direto do VMM sem camadas adicionais
Armazenamento de Snapshots	Sistema de arquivos local (ext4/xfs)	Caminho simples; backend S3 planejado

4 Metodologia Experimental

A metodologia do SnapVM segue um ciclo de desenvolvimento iterativo e orientado a experimentos. Como o projeto está em evolução, os resultados são tratados como preliminares, mas todos os dados apresentados derivam de execuções reais do MVP.

4.1 Questões de Pesquisa

Para orientar a avaliação, foram definidas as seguintes questões de pesquisa:

- **QP1:** snapshots reduzem o consumo de tokens e a latência de recuperação quando comparados a reparo manual ou rollback lógico em ambientes *stateful*?

- **QP2:** agentes de IA conseguem escolher autonomamente a restauração por snapshot quando essa ferramenta está disponível?
- **QP3:** checkpoints gerenciados pelo próprio agente melhoram a continuidade da tarefa e reduzem poluição de contexto?
- **QP4:** em falhas complexas, a restauração por snapshot mantém custo mais estável do que o reparo manual?

4.2 Ambiente Experimental

Os experimentos foram conduzidos com uma aplicação backend simples, composta por API Node.js/Express e banco PostgreSQL. O contrato de saúde do ambiente é verificado por um endpoint `/health`, que consulta o banco e valida a existência de estruturas esperadas, como a tabela `users` nos cenários iniciais. Falhas são injetadas de forma controlada pelo orquestrador, sem intervenção direta do agente, para simular corrupções e efeitos colaterais típicos de fluxos agênticos.

O agente utilizado nos experimentos com LLM segue um loop ReAct e foi baseado no modelo GPT-4o com temperatura 0. A escolha de temperatura 0 busca reduzir variabilidade, mas não elimina completamente diferenças de comportamento entre execuções. Por isso, os experimentos foram repetidos múltiplas vezes e os resultados são apresentados como médias.

4.3 Baselines Utilizados

Foram utilizados dois baselines principais:

- **Baseline lógico (Git):** representa rollback de arquivos por meio de `git reset --hard`. É usado como baseline mínimo, pois não restaura estado vivo de banco, processos ou memória.
- **Baseline físico (SnapVM):** representa restauração do ambiente a partir de snapshot, retornando a aplicação e o banco a um estado previamente capturado.

Nos experimentos com recuperação autônoma, também há uma comparação com o agente padrão, que tenta reparar manualmente a falha usando ferramentas como `query_db` e `execute_bash`.

4.4 Organização dos Experimentos

A Tabela 3 resume os experimentos conduzidos até o momento, sua função dentro da tese e o número de execuções utilizadas. Essa organização deixa explícito que os resultados são médias ou agregações de múltiplas execuções, e não observações pontuais.

4.5 Fases de Desenvolvimento e Cronograma

O desenvolvimento do SnapVM é organizado em quatro *sprints* sequenciais, alinhados ao calendário acadêmico e às entregas da disciplina de Trabalho Interdisciplinar 5 (TI5). A Tabela 4 detalha a distribuição temporal do projeto.

Table 3. Organização dos experimentos do SnapVM

Experimento	Tipo	Finalidade	Execuções
Exp. 1	Mock determinístico	Validar a falha do rollback lógico em estado <i>stateful</i> e medir baseline de infraestrutura.	10 iterações
Exp. 2	LLM real	Medir custo cognitivo de recuperação com Git vs. snapshot.	20 por baseline
Exp. 3	Otimização	Comparar snapshot completo e incremental em armazenamento e latência.	Medição direta
Exp. 4	Autônomo	Avaliar se o agente escolhe <code>restore_last_snapshot</code> sem coaching explícito.	20 por baseline
Exp. 4.1	Falhas complexas	Avaliar comportamento autônomo em cinco cenários multicamadas.	4 por cenário
Exp. 4.2	Límite superior	Medir restauração priorizada/forçada como benchmark controlado.	4 por cenário
Exp. 5	Checkpointing	Avaliar checkpoints proativos gerenciados pelo agente.	3 por baseline

Table 4. Cronograma de Sprints e Entregas (TI5 - 1º Semestre de 2026)

Período	Fase	Marco / Entrega da Disciplina
Fev – Mar/2026	Sprint 1	Validação de Conceitos (Exp 1 - Mock), Definição do Problema e Arquitetura.
Abr – Mai/2026	Sprint 2	Implementação da Orquestração, Integração Live LLM (Exps 2 a 5) e Apresentação Intermediária.
Mai – Jun/2026	Sprint 3	Refinamento de CLI, Health Probes Avançados e Otimização Incremental.
Jun – Jul/2026	Sprint 4	Exploração em Árvore (<i>Branching</i> - Exp 6), Escrita do Artigo Final e Defesa.

4.6 Critérios Gerais de Avaliação

O sucesso do sistema é avaliado contra critérios mensuráveis, preservando os objetivos técnicos do documento original:

- tempo de boot frio da VM inferior a 500 ms no hardware de referência;
- tempo de criação de snapshot inferior a 200 ms para VMs com até 256 MB de memória guest;
- tempo de restauração de snapshot inferior a 150 ms, excluindo recarga de memória do disco;
- fidelidade de estado: preservação do estado em memória e de descritores relevantes em ciclos de snapshot-restore;
- overhead de inicialização de clone inferior a 50 ms por instância em relação ao baseline de restauração.

5 Métricas e Variáveis de Análise

Para que os resultados sejam interpretáveis, as métricas utilizadas nos experimentos são definidas explicitamente na Tabela 5. A variável principal de análise é o **consumo de tokens**, usado como proxy de carga cognitiva do agente. A justificativa é que, quando o agente precisa reparar manualmente uma falha, ele tende a gerar diagnósticos, hipóteses, comandos, consultas e verificações adicionais. Quando uma restauração por snapshot substitui esse processo, parte do esforço de recuperação é transferida do raciocínio textual para uma ação operacional atômica.

A tese operacional avaliada é a seguinte: se snapshots funcionam como primitiva de raciocínio para agentes, então sua disponibilidade deve reduzir tokens e chamadas necessárias para recuperar ambientes *stateful*, pois a ação de restaurar substitui cadeias longas de diagnóstico e reparo.

Table 5. Métricas utilizadas na avaliação experimental

Métrica	Definição e interpretação
Consumo de tokens	Soma de tokens utilizados pelo agente durante a recuperação ou conclusão da tarefa. É a variável principal de análise e funciona como proxy de carga cognitiva.
Latência	Tempo necessário para recuperar o ambiente ou concluir a tarefa, medido em segundos.
Chamadas de ferramenta	Número de ações executadas pelo agente até atingir o estado correto ou encerrar o trial.
Poliuição de contexto	Volume de informações de erro, logs, comandos e tentativas de reparo acumuladas na janela de contexto.
Taxa de sucesso	Percentual de execuções que satisfazem os critérios de validação definidos pelo experimento.
Estratégia de recuperação	Sequência de ações escolhida pelo agente, classificada como reparo manual, restauração por snapshot, checkpointing ou falha.

6 Resultados Preliminares

Esta seção consolida os dados coletados nos experimentos executados até o momento. Como o projeto ainda está em evolução, os resultados são apresentados como preliminares, mas derivados de execuções reais. A análise privilegia a relação entre snapshots, carga cognitiva do agente e recuperação de ambientes *stateful*.

6.1 Experimento 1: Baseline Determinístico de Rollback Stateful

O Experimento 1 foi conduzido sem integração com LLM, utilizando uma máquina de estados determinística para isolar métricas de infraestrutura. O cenário consistiu na criação da tabela `users`, captura de milestone, injeção de falha via `DROP TABLE users;` e tentativa de rollback. A Tabela 6 apresenta as médias obtidas em 10 iterações.

Table 6. Experimento 1: métricas determinísticas de rollback (10 iterações)

Baseline	Capture	Restore	Storage	Penalty	Contrato
Git	0.008s	0.006s	27.420B	0.004s	PASS após rotina de penalidade
Firecracker Snapshot	0.413s	0.158s	268.449.211B	0.000s	PASS após restauração

O resultado central do Experimento 1 é que o `git reset` reverteu arquivos, mas não restaurou o estado do banco já corrompido. Assim, foi necessária uma rotina de penalidade para reparar o ambiente. A restauração por snapshot, por outro lado, retornou o banco ao estado capturado. Esse experimento não pretende provar comportamento agentic, mas estabelecer a diferença entre rollback lógico e restauração física de estado.

6.2 Experimento 2: Recuperação com LLM Real

O Experimento 2 substituiu o mock determinístico por um agente ReAct baseado em GPT-4o. Cada baseline foi executado em 20 iterações independentes. O agente recebeu ferramentas para consultar o banco, executar comandos e verificar o endpoint `/health`. Após a falha, o baseline Git exigiu reparo manual do banco; no baseline SnapVM, a restauração do snapshot retornou imediatamente o ambiente ao estado saudável.

A redução de tokens é o dado mais relevante para a tese deste artigo. Ela sugere que, quando o ambiente é fisicamente restaurado, o agente deixa de gastar recursos cog-

Table 7. Experimento 2: eficiência de recuperação com LLM real (20 iterações por baseline)

Métrica Média	Git (Standard)	SnapVM	Diferença
Capture Latency	0.030s	0.484s	maior custo de captura
Restore Latency (Total)	6.572s	0.184s	-97%
Penalty Time (LLM Repair)	6.570s	0.000s	-100%
Consumo de Tokens	2860.3	549.2	-80.8% (5.2x)
Poliuição de Contexto	456.1	165.6	-63.6% (2.7x)
Storage Overhead	27.4 KB	268.4 MB	maior custo de armazenamento

nitivos diagnosticando e reparando manualmente uma corrupção *stateful*. Em outras palavras, a restauração por snapshot substituiu uma sequência de raciocínio corretivo por uma operação ambiental atômica.

6.3 Experimento 3: Otimização Incremental de Snapshots

O Experimento 3 investigou o uso de snapshots incrementais para reduzir custo de armazenamento e viabilizar checkpoints frequentes. A motivação surgiu do próprio Experimento 2: embora snapshots reduzam tokens e latência de reparo, snapshots completos possuem custo físico maior que commits Git. A Tabela 8 resume os dados de comparação entre snapshots completos e incrementais.

Table 8. Experimento 3: comparação entre snapshots completos e incrementais

Tipo de Snapshot	Latência de Captura	Tamanho	Observação
Full	0.432s	268.4 MB	Captura completa do estado de memória.
Diff	0.350s	170.5 MB	Redução de 19% em latência e 36.5% em armazenamento.

Esse resultado é infraestrutural, mas relevante para agentes: se checkpoints forem baratos o suficiente, o agente pode salvar estados com maior frequência antes de operações arriscadas. Assim, o Experimento 3 sustenta a viabilidade técnica de transformar snapshots em uma ferramenta recorrente no ciclo agentic.

6.4 Experimento 4: Recuperação Autônoma Sem Coaching

O Experimento 4 avaliou se o agente escolheria espontaneamente a ferramenta `restore_last_snapshot` quando ela estivesse disponível. Diferentemente do Experimento 2, não houve instrução explícita para reparar o banco; o agente recebeu apenas uma percepção neutra sobre alteração de saúde do ambiente. Foram executadas 20 iterações por baseline, totalizando 40 trials.

Table 9. Experimento 4: recuperação autônoma sem coaching (20 iterações por baseline)

Métrica	Standard (Manual)	SnapVM (Restore)	Delta
Success Rate	100%	100%	-
Avg Tool Calls	2.2	1.6	-27%
Avg Token Consumption	1298.8	776.6	-40%
Avg Recovery Latency	2.61s	1.72s	-34%
Avg Context Pollution	259.6 tokens	248.9 tokens	-4%

Esse experimento é central para a tese do artigo. O agente não apenas se beneficiou de snapshots quando a restauração foi imposta; ele escolheu a ferramenta em 85% das execuções. Ao mesmo tempo, os 15% de reparo manual indicam que a disponibilidade da ferramenta não garante sua adoção

Table 10. Distribuição de estratégia no baseline SnapVM do Experimento 4

Estratégia	Ocorrências	Percentual
Snapshot Restore	17/20	85%
Manual Repair	3/20	15%

universal. A decisão do agente depende da forma como a falha é percebida e de como as ferramentas são descritas.

6.5 Experimento 4.1: Falhas Complexas e Decisão Autônoma

O Experimento 4.1 ampliou o cenário para falhas multicadas, avaliando cinco tipos de corrupção com quatro execuções por cenário. O objetivo foi observar se a autonomia do agente persistiria quando o erro não fosse uma simples tabela ausente. A Tabela 11 apresenta o resultado geral.

Table 11. Experimento 4.1: resultados gerais em falhas complexas

Métrica	Standard (Manual)	SnapVM (Autônomo)	Delta
Success Rate	100%	100%	–
Avg Tool Calls	2.2	1.9	-14%
Avg Token Consumption	1349.5	1174.1	-13%
Avg Recovery Latency	2.85s	2.14s	-25%
Avg Context Pollution	276.1 tokens	263.3 tokens	-5%

Table 12. Experimento 4.1: detalhamento por cenário

Cenário	Std Calls	Snap Calls	Std Tokens	Snap Tokens	Estratégia Snap
F1: Multi-table	3.0	3.0	1751	1688	50% Restore / 50% Manual
F2: Process & Config	3.0	2.0	1758	1036	75% Restore / 25% Manual
F3: Schema & Data	1.0	1.0	740	818	100% Manual
F4: Permissions	3.0	2.5	1757	1514	50% Restore / 50% Manual
F5: Cascade	1.0	1.0	742	816	100% Manual

O achado mais importante é comportamental. Em falhas complexas, o agente escolheu reparo manual em grande parte dos casos, especialmente quando a falha parecia ter uma correção local. Isso mostra que a eficiência do SnapVM depende de dois fatores: a capacidade da infraestrutura restaurar rapidamente o estado e a política de decisão do agente escolher essa restauração no momento adequado.

6.6 Experimento 4.2: Restauração Forçada como Limite Superior

O Experimento 4.2 removeu a variável de escolha autônoma ao forçar o agente a priorizar restauração por snapshot. Assim, ele não mede autonomia, mas sim o limite superior controlado da abordagem quando a política de decisão escolhe restauração como caminho preferencial. A Tabela 13 resume os resultados gerais.

Os dados sugerem que, em cenários controlados, a restauração por snapshot aproxima a recuperação de um custo estável, independente da complexidade aparente da falha. Entretanto, por se tratar de restauração priorizada, esse experimento deve ser interpretado como benchmark de limite superior, não como prova de comportamento autônomo.

6.7 Experimento 5: Checkpointing Proativo

O Experimento 5 investigou o uso de checkpoints gerenciados pelo próprio agente durante uma tarefa mais longa de

Table 13. Experimento 4.2: restauração forçada como benchmark controlado

Métrica	Standard (Manual)	SnapVM (Forced Restore)	Delta
Success Rate	100%	100%	–
Avg Tool Calls	2.2	1.0	-55%
Avg Token Consumption	1350.2	441.8	-67%
Avg Recovery Latency	3.25s	0.92s	-72%
Avg Context Pollution	275.1 tokens	266.8 tokens	-3%

Table 14. Experimento 4.2: custo por cenário em falhas complexas

Cenário	Std Calls	Snap Calls	Std Tokens	Snap Tokens	Ganho
F1: Corrupção Multi-tabela	3.0	1.0	1758.5	449.0	3.9x
F2: Queda Processo + Config	3.0	1.0	1758.5	449.0	3.9x
F3: Corrupção Schema + Dados	1.0	1.0	740.5	431.0	1.7x
F4: Revogar Permissões	3.0	1.0	1757.0	449.0	3.9x
F5: Falha em Cascata	1.0	1.0	736.8	431.0	1.7x

migração. Foram executados três trials por baseline. Esse experimento é conceitualmente importante porque desloca o snapshot de uma ferramenta reativa para uma primitiva proativa de desenvolvimento.

Table 15. Experimento 5: checkpointing proativo gerenciado pelo agente

Métrica	Baseline Standard	Baseline Checkpoint	Delta
Task Completion Rate	67%	100%	+33%
Avg Tool Calls	24.3	26.7	+10%
Avg Token Consumption	47553	42060	-12%
Avg Task Latency	92.0s	64.6s	-30%
Avg Context Pollution	3124 tokens	1590 tokens	-49%

Apesar da amostra menor, o Experimento 5 é o mais alinhado à tese de snapshots como primitiva de raciocínio. O agente com checkpoint não apenas se recuperou após falhas; ele salvou estados antes de etapas críticas, preservou progresso e reduziu a quantidade de contexto contaminado por tentativas de reparo. Os labels utilizados pelo agente incluíram *initial_state_before_migration*, *Products table created and seeded* e *Categories table created and products assigned categories*.

6.8 Síntese Comparativa dos Resultados

A Tabela 16 resume os principais ganhos observados nos experimentos mais diretamente ligados à tese agentic.

7 Discussão

Os resultados preliminares indicam que o valor do SnapVM não está apenas em restaurar ambientes rapidamente, mas em reduzir a quantidade de raciocínio corretivo exigido de agentes de IA. Essa seção discute as principais implicações dos experimentos.

7.1 Consumo de Tokens como Proxy de Carga Cognitiva

Neste trabalho, o consumo de tokens é interpretado como proxy de carga cognitiva do agente. Quando o agente precisa reparar manualmente um ambiente corrompido, ele tende a acumular consultas, diagnósticos, hipóteses, comandos e verificações. Ao restaurar um snapshot, parte desse processo é substituída por uma ação operacional atômica. A redução de 80.8% no Experimento 2, 40% no Experimento 4 e 67%

Table 16. Síntese dos resultados preliminares relacionados à carga cognitiva do agente

Experimento	Foco	Resultado principal	Interpretação
Exp. 2	LLM + rollback	-80.8% tokens	Snapshot reduz reparo cognitivo após falha <i>stateful</i> .
Exp. 4	Autonomia	85% restore	Agente incorpora snapshot como ação de recuperação.
Exp. 4.1	Falhas complexas	-13% tokens	Escolha do agente limita ganhos em cenários ambíguos.
Exp. 4.2	Limite superior	-67% tokens	Restauração priorizada estabiliza custo de recuperação.
Exp. 5	Checkpointing	-49% contexto	Checkpoints funcionam como memória operacional reversível.

no Experimento 4.2 sugere que snapshots podem diminuir o esforço de raciocínio necessário para recuperar ambientes *stateful*.

7.2 Snapshot como Ferramenta de Decisão

O Experimento 4 mostra que o snapshot pode ser incorporado ao espaço de ações do agente. A escolha espontânea de `restore_last_snapshot` em 85% das execuções indica que, quando a ferramenta está disponível e a falha é percebida como degradação de estado, o agente tende a reconhecê-la como uma opção eficiente de recuperação. Isso sustenta a tese de que snapshots podem deixar de ser apenas mecanismos de infraestrutura e passar a funcionar como primitivas no ciclo *Reason-Act-Observe*.

7.3 Infraestrutura Eficiente vs. Comportamento do Agente

O Experimento 4.1 mostra que a eficiência da infraestrutura não garante, por si só, o melhor comportamento agêntico. Em falhas complexas, o agente frequentemente escolheu reparar manualmente quando o erro parecia local ou facilmente corrigível. Esse comportamento reduz os ganhos possíveis, mas também revela um ponto importante: arquiteturas futuras precisam de políticas de decisão que ajudem o agente a estimar quando restaurar e quando reparar. Assim, o problema passa a ser tanto de infraestrutura quanto de desenho de ferramentas, prompts e critérios de custo.

7.4 Checkpoints como Memória Operacional Reversível

O Experimento 5 sugere um uso mais avançado de snapshots: checkpointing proativo. Nesse cenário, o agente não apenas retorna depois de uma falha, mas cria pontos seguros antes de ações arriscadas. Essa capacidade aproxima snapshots de uma memória operacional reversível: o agente preserva progresso, testa ações, descarta estados ruins e continua a partir de pontos confiáveis. A redução de 49% em poluição de contexto reforça a ideia de que checkpoints podem manter o raciocínio do agente mais limpo e orientado ao progresso.

7.5 Implicações para Arquiteturas Agênticas

A principal implicação do SnapVM é que ambientes de execução para agentes não devem ser vistos apenas como sandboxes passivas. Eles podem oferecer capacidades ativas de gerenciamento de estado. Em vez de depender exclusivamente do modelo para lembrar, diagnosticar e desfazer ações, a infraestrutura pode fornecer mecanismos explícitos de retorno, checkpointing e, futuramente, ramificação. Essa visão

aproxima o ambiente de execução de um componente cognitivo externo, capaz de reduzir erros acumulados e permitir exploração controlada.

8 Limitações e Ameaças à Validade

Como o projeto está em evolução, os resultados devem ser interpretados com cautela. As principais ameaças à validade são:

- **Uso de um único modelo:** os experimentos foram conduzidos com GPT-4o em temperatura 0. Outros modelos podem apresentar estratégias diferentes de reparo, restauração ou checkpointing.
- **Cenários isolados e controlados:** os workloads atuais se concentram em uma aplicação backend com PostgreSQL. Ainda é necessário avaliar filas, autenticação, falhas de rede, múltiplos serviços e sistemas distribuídos.
- **Amostra reduzida no Experimento 5:** embora os resultados de checkpointing sejam promissores, o V5 teve apenas três execuções por baseline, sendo necessário ampliar a amostra.
- **Resultados preliminares:** os dados são reais, mas não finais. O projeto ainda passa por evolução de arquitetura, instrumentação e desenho experimental.
- **Influência de prompts e descrições de ferramentas:** a decisão do agente pode mudar conforme a forma como a falha é apresentada e como as ferramentas são descritas.
- **Baseline Git mínimo:** o Git não é tratado como concorrente direto de snapshots, mas como referência comum de rollback lógico. Comparações futuras devem incluir outros mecanismos de checkpoint e sandboxing.

9 Riscos e Mitigações

Além das ameaças à validade, o desenvolvimento do SnapVM envolve riscos técnicos já identificados:

- **Disponibilidade de KVM em hardware CI/CD:** mitigado direcionando a execução para ambientes bare-metal ou com virtualização aninhada habilitada; modo de fallback com QEMU pode ser usado para desenvolvimento local.
- **Crescimento do tamanho de snapshots de memória:** mitigado pela avaliação de snapshots incrementais, compressão e snapshots delta em relação a uma imagem base.
- **Estado de rede após restaurações:** o estado completo de rede permanece fora do escopo das fases iniciais; checkpoint completo de rede é planejado para fases posteriores.
- **Decisão subótima do agente:** mitigado por melhoria de descrições de ferramentas, políticas custo-aware e prompts que orientem quando restaurar ou reparar.

10 Trabalhos Futuros

Como o projeto se encontra em desenvolvimento contínuo, os próximos passos buscam ampliar a robustez experimental e aproximar o SnapVM de uma primitiva geral para arquiteturas agentic.

10.1 Ampliação da Amostra e Análise Estatística

Os experimentos futuros devem aumentar o número de execuções, especialmente no Experimento 5, e incluir métricas de dispersão como desvio padrão, mediana, mínimo, máximo e intervalos de confiança. Isso permitirá avaliar se as médias observadas são estáveis ou influenciadas por outliers.

10.2 Avaliação com Múltiplos Modelos

A avaliação atual usa GPT-4o com temperatura 0. Trabalhos futuros devem comparar diferentes modelos de linguagem, incluindo alternativas proprietárias e abertas, para verificar se o uso de snapshots como ferramenta é generalizável.

10.3 Novos Workloads Stateful

O ambiente atual usa API Express e PostgreSQL. Pretende-se expandir a avaliação para cenários com filas, autenticação, serviços distribuídos, múltiplos bancos, caches e falhas de rede. Esses cenários permitirão testar se o ganho de snapshots cresce conforme aumenta a complexidade do ambiente.

10.4 Políticas de Decisão Custo-Aware

O Experimento 4.1 mostrou que agentes podem insistir em reparo manual mesmo quando a restauração seria mais eficiente. Uma direção futura é tornar o custo de tokens, latência e risco de efeitos colaterais explícito para o agente, orientando políticas de decisão entre reparar, restaurar ou salvar checkpoint.

10.5 Experimento 6: ReSnapAct e Branching Especulativo

A Sprint 4 introduzirá o Experimento 6, baseado no padrão *ReSnapAct*: Reason → Save → Act → Observe → Restore/Branch → Retry. O objetivo é validar se agentes com capacidade de ramificação via snapshots podem explorar múltiplas estratégias de debugging especulativo em falhas ambíguas, descartando tentativas falhas e preservando a janela de contexto. Adicionalmente, planeja-se explorar backend de armazenamento S3 para distribuição de snapshots entre múltiplos agentes.

11 Conclusão

Este artigo apresentou o SnapVM como uma investigação experimental sobre snapshots como primitiva operacional para agentes de IA em ambientes *stateful*. A proposta desloca

snapshots do papel tradicional de mecanismo de infraestrutura para o papel de ferramenta disponível ao agente, permitindo restauração, checkpointing e, futuramente, ramificação de estados.

Os resultados preliminares indicam que snapshots podem reduzir consumo de tokens, latência, chamadas de ferramenta e poluição de contexto em diferentes cenários controlados. No Experimento 2, a restauração por snapshot reduziu o consumo de tokens em 80.8% em relação ao baseline Git. No Experimento 4, o agente escolheu autonomamente restaurar snapshots em 85% das execuções, com redução de 40% no consumo de tokens. No Experimento 5, checkpoints proativos indicaram aumento da taxa de conclusão e redução expressiva da poluição de contexto, embora ainda com amostra reduzida.

Como o projeto está em evolução, esses resultados não devem ser interpretados como conclusivos. Ainda assim, eles sugerem uma direção promissora: ambientes reversíveis podem funcionar como memória operacional externa para agentes autônomos. Em vez de apenas executar comandos e tentar reparar falhas, o agente pode preservar estados, retornar a pontos confiáveis e explorar alternativas de forma controlada. Essa visão abre caminho para arquiteturas agentic mais robustas, previsíveis e eficientes em ambientes computacionais mutáveis.

12 Disponibilidade de Código / Repositório

O código-fonte, a documentação e os experimentos iniciais do projeto SnapVM estão disponíveis publicamente no repositório oficial da disciplina no GitHub Classroom, que pode ser acessado através do seguinte link: <https://github.com/ICEI-PUC-Minas-CC-TI/plmg-cc-ti5-2026-1-snapvm/tree/main>.

References

- Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association.
- CRIU Project (2012). CRIU: Checkpoint/restore in userspace. <https://criu.org>. Acesso em: março 2026.
- E2B (2023). E2B: Cloud runtime for AI agents. <https://e2b.dev>. Acesso em: março 2026.
- Jain, M. (2020). Study of Firecracker MicroVM. *arXiv preprint arXiv:2005.12821*.
- Kata Containers Contributors (2017). Kata containers: The speed of containers, the security of VMs. <https://katacontainers.io>. Acesso em: março 2026.
- Lagar-Cavilla, H. A., Whitney, J. A., Scannell, A., Patchin, P., Rumble, S. M., de Lara, E., Brudno, M., and Satyanarayanan, M. (2009). SnowFlock: Rapid virtual machine cloning for cloud computing. In *Proceedings of the*

- 4th ACM European Conference on Computer Systems (EuroSys '09)*, pages 1–12. ACM.
- Long, J., Tai, H.-Y., Hsieh, S.-T., and Yuan, M. J. (2020). A lightweight design for serverless Function-as-a-Service. *arXiv preprint arXiv:2010.07115*.
- Weissman, Z., Tiemann, T., Eisenbarth, T., and Sunar, B. (2023). Microarchitectural security of AWS Firecracker VMM for serverless cloud platforms. *arXiv preprint arXiv:2311.15999*.
- Young, E. G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A. C., and Arpaci-Dusseau, R. H. (2019). The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. USENIX Association.